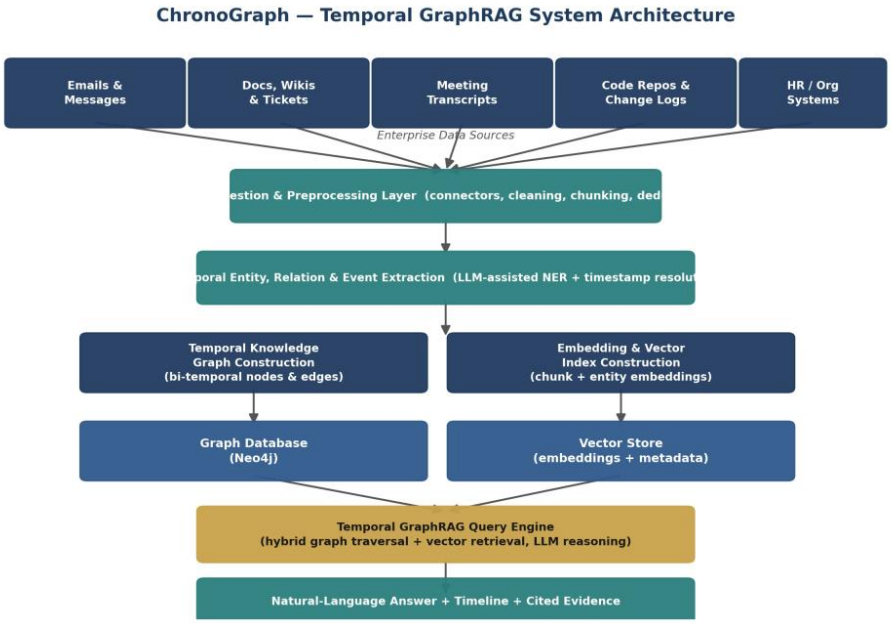
ChronoGraph

Temporal GraphRAG for Organizational History — Project Documentation

ChronoGraph converts fragmented enterprise data — documents, tickets, chats, meeting notes, and change records — into a timeaware knowledge graph, so users can query organizational history in natural language. It combines GraphRAG (grounding LLM answers in a structured graph, not just text chunks) with temporal modeling (tracking when facts became true, changed, or expired), producing answers that are timeline-aware and cited to source evidence.

1. Architecture

Four stages: ingestion, extraction, storage, and retrieval/generation.



Component	Responsibility
Ingestion	Connectors pull data from source systems; text is cleaned, deduplicated, chunked with source + timestamp metadata.
Extraction	LLM-assisted entity/relation extraction; resolves timestamps for each fact.
Temporal Graph	Entities and relationships stored as versioned nodes/edges with validity intervals (Neo4j).
Vector Index	Embeddings of source chunks and entity summaries for semantic search.
Query Engine	Decomposes the question, retrieves graph + vector context, orchestrates the LLM response.

2. Knowledge Graph Design

Core node types: Person, Technology, Decision, Project, Event, Document. Core relationships:

Relationship	Connects	Temporal Attributes
WORKED_ON / OWNED	Person → Project / Technology	valid_from, valid_to
MADE_DECISION	Person → Decision	decided_on
REPLACED_BY	Technology → Technology	changed_on
EVIDENCED_BY	Any node → Document	extracted_on

Modeling is bi-temporal: valid time (when a fact was true) is tracked separately from recorded_at (when ChronoGraph learned it). Superseded edges are closed, not deleted, so full history stays queryable.

3. Query Engine

- Question decomposition — identify entities, relationship types, and time constraints.

- Hybrid retrieval — graph traversal for structured facts + vector search for narrative context.
- Grounded generation — LLM answers with every claim traceable to a source Document via EVIDENCED_BY.
- Timeline construction — historical questions return an ordered, dated sequence of events.

4. Proposed Tech Stack

Layer	Technology
Graph DB	Neo4j (temporal properties, Cypher traversal)
Vector Store	Managed vector DB / Neo4j vector index
Orchestration	LangChain / LlamaIndex
LLM	Anthropic Claude — extraction & grounded generation
API / Frontend	FastAPI + React query console

5. Use Cases

- Onboarding — “Why did we move off Vendor X, and who made that call?”
- Audit — “Who owned the payments system between March and June 2022?”
- Incident review — “What changed in the pipeline in the two weeks before the outage?”

6. Evaluation, Risks & Roadmap

- Evaluate: extraction precision/recall, temporal accuracy, retrieval relevance, answer faithfulness, citation accuracy.
- Key risks: ambiguous relative timestamps, cross-source entity resolution, conflicting facts, LLM hallucination — mitigated by anchoring dates to metadata, alias tracking, preserving conflicting facts with source confidence, and requiring citations on every claim.
- Roadmap: multi-hop temporal reasoning, automated conflict surfacing, user-feedback loop, more connectors, permission-aware querying.